

Sincronizzazione e Clock Logici

Lamport, ordine causale e temporalita' distribuita

Architetture dei Sistemi Distribuiti

Lezione 20

Perche' Il Tempo Diventa Difficile

In un processo singolo il tempo sembra naturale:

- un'istruzione viene prima di un'altra;
- lo stato evolve lungo una sola linea;
- il log locale e' quasi sempre sufficiente.

In un sistema distribuito:

- ogni nodo ha il proprio clock;
- i messaggi hanno latenza variabile;
- due eventi possono essere indipendenti;
- non esiste un osservatore globale immediato.

Non chiediamo solo:

che ora era?

Chiediamo:

quale ordine possiamo affermare con certezza?

Domande tipiche:

- questo update ha causato quell'altro?
- questi due eventi sono concorrenti?
- tutti i nodi possono processare gli eventi nello stesso ordine?
- un lease e' davvero scaduto?

Scenario: Log Impossibile

A 10:00:00.200 send m to B

B 10:00:00.100 receive m from A

Domande:

- B ha ricevuto il messaggio prima dell'invio?
- oppure i clock fisici non sono allineati?
- quale informazione ci manca?

Questo scenario si riproduce con:

```
python3 labs/logical_clocks/physical_clock_skew.py
```

Ogni nodo puo' usare il proprio orologio. Utile per:

- log leggibili;
- timeout;
- misure approssimate;
- audit umano.

Ma presenta problemi:

- clock skew;
- clock drift;
- salti di orologio;
- ordine nei log non necessariamente causale.

Meccanismi come NTP o PTP riducono l'errore tra clock. Sono importanti per:

- lease;
- scadenze;
- SLA;
- audit;
- misure di latenza.

Pero' il risultato non e' un punto perfetto:

il tempo reale e' dentro un intervallo di incertezza

Happened-Before

Lamport propone una relazione causale:

$$a \rightarrow b$$

significa:

$$a \text{ happened-before } b$$

Regole:

- ordine locale nello stesso processo;
- invio messaggio prima della ricezione;
- transitività'.

Se non vale $a \rightarrow b$ e non vale $b \rightarrow a$, gli eventi sono concorrenti.

Ogni processo mantiene un contatore. Regole:

- evento locale: incrementa;
- send: incrementa e allega timestamp;
- receive: $\max(\text{locale}, \text{ricevuto}) + 1$.

Garanzia:

$$a \rightarrow b \Rightarrow L(a) < L(b)$$

Limite:

$$L(a) < L(b) \not\Rightarrow a \rightarrow b$$

Esempio Lamport

A: local event	$L=1$
A: send m to B	$L=2$
B: local event	$L=1$
B: receive m	$L=\max(1,2)+1=3$
B: local event	$L=4$

Il clock cresce rispettando la causalita' introdotta dal messaggio. Da provare:

```
python3 labs/logical_clocks/lamport_clock_simulation.py
```

A volte serve ordinare tutti gli eventi, anche concorrenti. Si usa:

$$(lamport_time, process_id)$$

Esempio:

$$(4, A) < (4, B)$$

se decidiamo che $A < B$. Questo ordine e' utile, ma artificiale: non dimostra causalita'.

Quando Serve L'Ordine Totale

Esempi:

- merge deterministico di log distribuiti;
- mutua esclusione distribuita;
- code di richieste;
- scelta deterministica tra update concorrenti.

Da provare:

```
python3 labs/logical_clocks/total_order_lamport.py
```

Domanda: stiamo scoprendo un ordine reale o imponendo un ordine utile?

Esempio Più Ricco: Mutua Esclusione

Tre processi chiedono accesso alla stessa risorsa:

B: request kv:x at L=1

A: request kv:x at L=1

C: request kv:x at L=1

Tutti possono ordinare nello stesso modo usando:

`(lamport_time, process_id)`

Da provare:

```
python3 labs/logical_clocks/lamport_mutex.py
```

Se due eventi sono concorrenti, Lamport puo' comunque assegnare:

$$L(a) < L(b)$$

Ma questo non significa:

$$a \rightarrow b$$

Quindi Lamport e' ottimo per rispettare causalita' nota, ma non basta per riconoscere sempre la concorrenza.

Con tre processi:

$$[A, B, C]$$

Ogni processo:

- incrementa la propria componente;
- invia il vettore nei messaggi;
- alla ricezione fa il massimo componente per componente.

I vector clock permettono di distinguere:

- causalita';
- ordine inverso;
- concorrenza.

Esempio Vector Clock

$A1 = \{ 'A': 1, 'B': 0, 'C': 0 \}$

$B1 = \{ 'A': 0, 'B': 1, 'C': 0 \}$

I due vettori non sono confrontabili:

gli eventi sono concorrenti

Da provare:

```
python3 labs/logical_clocks/vector_clock_simulation.py
```

Un sistema di causal delivery deve evitare di consegnare un messaggio prima dei messaggi da cui dipende causalmente. Esempio:

- m1: A pubblica x;
- m2: B reagisce a x;
- C riceve m2 prima di m1.

C deve bufferizzare m2 finché m1 non è consegnabile.

Da eseguire:

```
python3 labs/logical_clocks/causal_delivery.py
```

Osservare:

- arrivo e consegna non coincidono;
- il buffer serve a rispettare la causalita';
- il metadata temporale condiziona il comportamento del sistema.

Forma	Quando ha senso
Clock fisico locale	log, timeout, misure approssimate
Clock fisico sincronizzato	lease, scadenze, audit
Lamport clock	causalita' tra messaggi
Ordine totale Lamport	code, lock distribuiti, log merge
Vector clock	conflitti concorrenti e causalita' precisa
Coordinatore	sequenze globali semplici
Quorum/consenso	decisioni critiche e log replicati

Collegamento Con Il KV Store

I clock aiutano a progettare:

- update replicati;
- risoluzione dei conflitti;
- read repair;
- causal read;
- debug della capstone;
- log distribuiti ordinabili.

Esempio:

SET x 1 timestamp=(7,A)

SET x 2 timestamp=(7,B)

Un ordine totale sceglie un vincitore. Un vector clock puo' rivelare concorrenza.

Prima di scegliere un timestamp chiedere:

- ❶ voglio misurare durata reale?
- ❷ voglio rispettare causalita' ?
- ❸ voglio imporre un ordine totale?
- ❹ voglio rilevare conflitti concorrenti?
- ❺ quanto metadata posso permettermi?
- ❻ la membership e' stabile o dinamica?

I clock rispondono a domande di ordine. Ma a volte serve una decisione condivisa:

- chi e' il leader?
- quale comando entra nella prossima posizione del log?
- quale configurazione del cluster e' valida?
- quale valore viene scelto tra proposte concorrenti?

Qui non basta un timestamp: serve consenso.

Paxos single-decree risolve il consenso su un solo valore. Problema:

piu' nodi devono decidere lo stesso valore nonostante messaggi ritardati, nodi fermi e proposer concorrenti

Esempi di valore:

- leader corrente;
- prossima entry del log;
- configurazione valida del cluster;
- comando scelto tra proposte concorrenti.

Tre acceptor, quorum di due:

- A1 e A2 accettano leader=P1;
- A2 e A3 accettano leader=P2.

Due osservatori potrebbero concludere:

$\{A1, A2\}$ ha scelto P1

$\{A2, A3\}$ ha scelto P2

Questo e' lo stato vietato: due valori scelti.

Paxos deve garantire:

- **Validity**: solo valori proposti possono essere scelti;
- **Agreement**: non possono essere scelti due valori diversi;
- **Learnability**: chi apprende una decisione corretta apprende il valore scelto.

La proprietà centrale per noi è:

Agreement

È ciò che impedisce split brain logici e doppie entry nella stessa posizione di log.

La soluzione combina tre idee:

- ① quorum che si intersecano;
- ② proposal number ordinabili e unici;
- ③ memoria del passato riportata dagli acceptor.

Il proposer piu' nuovo puo' superare proposte vecchie. Ma se scopre che un valore potrebbe gia' essere stato scelto, deve conservarlo.

- **Proposer:** propone un valore usando un proposal number.
- **Acceptor:** promette e accetta proposte secondo regole precise.
- **Learner:** scopre quale valore e' stato scelto.

Ogni acceptor conserva:

`promised_n`

`accepted_n, accepted_value`

Definiamo:

- A: insieme degli acceptor;
- Q: insieme dei quorum;
- N: proposal number;
- V: valori proponibili.

Per ogni acceptor a:

- `promised[a]` in N or none
- `accepted[a]` in (N x V) or none

Un valore v è scelto se un quorum ha accettato la stessa coppia (n, v) .

Quorum E Intersezione

Con tre acceptor:

$$quorum = 2$$

Due quorum di maggioranza si intersecano sempre. Questa intersezione e' la ragione tecnica per cui:

- una nuova proposta puo' scoprire cosa era gia' stato accettato;
- un valore potenzialmente scelto non viene perso;
- due valori diversi non possono essere scelti indipendentemente.

Formalmente:

for every q_1, q_2 in Q : $q_1 \cap q_2 \neq \text{empty}$

Ogni proposta ha un identificatore ordinabile:

`(round, proposer_id)`

Non e' un clock fisico. Serve a dire:

- questa proposta e' piu' nuova di quella promessa prima;
- un acceptor puo' rifiutare proposte vecchie;
- un proposer piu' nuovo deve rispettare valori gia' accettati.

Tre invarianti reggono la safety:

- `promised[a]` non diminuisce mai;
- un acceptor accetta (n, v) solo se non ha promesso un numero maggiore;
- un proposer che vede valori già accettati deve riproporre quello con `accepted_n` massimo.

Questi invarianti fanno viaggiare la memoria delle decisioni potenziali da un quorum al successivo.

Fase 1: Prepare / Promise

Il proposer invia:

PREPARE(n)

L'acceptor risponde:

PROMISE(n , last_accepted)

se non ha già promesso una proposta più alta. La promise significa:

non accettero' proposte con numero minore di n

Fase 2: Accept / Accepted

Se il proposer ottiene promise da un quorum:

- se nessuno aveva accettato valori, propone il proprio valore;
- se qualcuno aveva accettato valori, deve riproporre quello con proposal number piu' alto.

Poi invia:

ACCEPT(n, value)

Quando un quorum accetta lo stesso valore, quel valore e' scelto.

Perche' Non Si Puo' Ignorare Il Passato

Supponiamo:

- P1 ha fatto accettare v1 a un quorum;
- P1 cade prima di comunicarlo a tutti;
- P2 arriva con proposal number piu' alto e vorrebbe proporre v2.

Paxos obbliga P2 a chiedere agli acceptor cosa avevano gia' accettato. Se scopre v1, deve riproporre v1.

Safety

- al massimo un valore viene scelto;
- la proprieta' vale anche con proposer concorrenti e messaggi ritardati.

Liveness

- Paxos base puo' non progredire se proposer concorrenti continuano a disturbarsi;
- nella pratica si usa spesso un leader stabile;
- Multi-Paxos riusa il leader per molte posizioni di log.

Supponiamo che v_1 sia scelto da un quorum q . Ogni proposta successiva che ottiene promise da un quorum q' deve intersecare q . Quindi almeno un acceptor puo' riportare la memoria di v_1 . La regola della fase 2 obbliga il nuovo proposer a conservare il valore con `accepted_n` massimo. Conclusione: una proposta successiva non puo' scegliere liberamente un valore diverso da quello gia' potenzialmente scelto.

Lab: Paxos Single-Decree

Da eseguire:

```
python3 labs/logical_clocks/paxos_single_decree.py
```

Osservare:

- due proposer partono con valori diversi;
- il secondo ha proposal number piu' alto;
- se scopre un valore gia' accettato, deve conservarlo;
- la safety impedisce due decisioni diverse.

Il tempo distribuito non e' un oggetto unico. E' una famiglia di contratti:

- tempo fisico per scadenze;
- tempo logico per causalita';
- ordine totale per decidere;
- vector clock per riconoscere concorrenza.
- consenso per scegliere un valore condiviso.

La domanda corretta non e':

che timestamp uso?

ma:

quale proprieta' temporale voglio promettere?